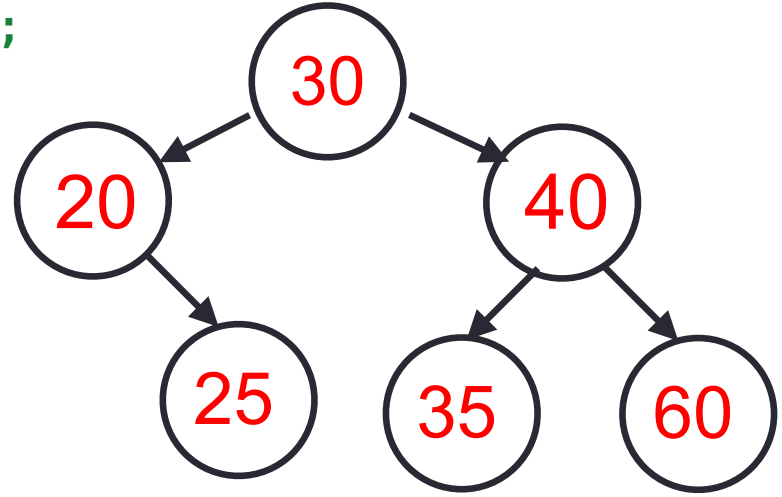
ITERATORS
TREE TRAVERSALS

Problem Solving with Computers-II

Join iclicker at **https://join.iclicker.com/GEXG**

Traversing C++ STL std::set

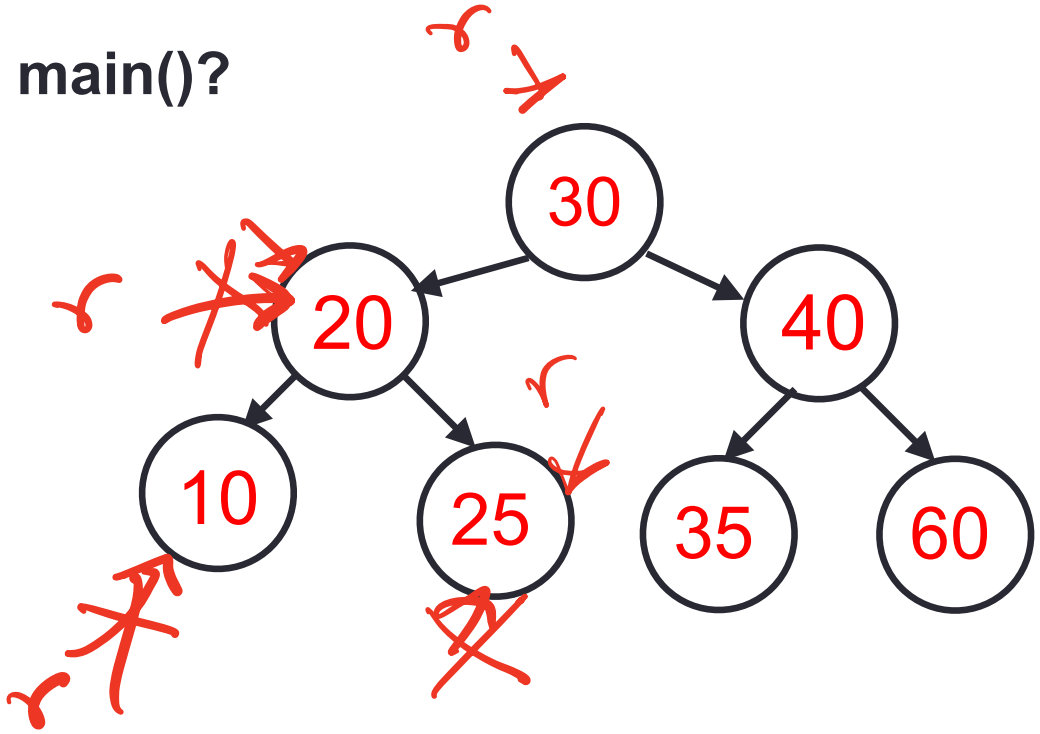
```
std::set<int> s = {30, 20, 25, 40,35, 60};

for (int x : s) {
    cout << x << " ";
}
```



Why doesn’t std::set have a printInorder() function?

Why not just write this code in main()?

```
Node* r = b.min(root);
while(r){
    cout << r->data << " ";
    r = b.successor(r);
}
```



10 , 20 , 25 , 30 . . . . .

Our goal: Implement iterator for custom BST

*it

What you write…

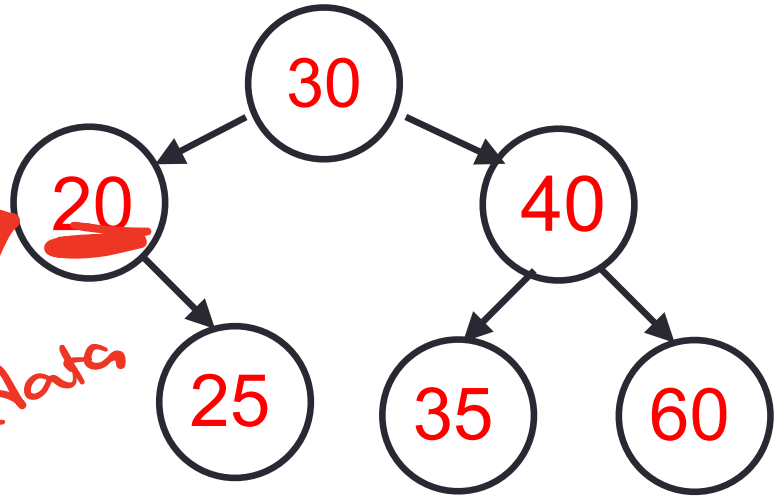
```
std::set<int> s = {. . .}
for (int x : s) {
     cout << x << " ";
}
```

this

it

curr

curr->data



What actually happens:

```
for (std::set<int>::iterator it = s.begin(); it != s.end(); ++it) {
    cout << *it << " ";
}
```

An iterator is an object that behaves like a pointer – it points to an element in a container, you can dereference it and advance it.

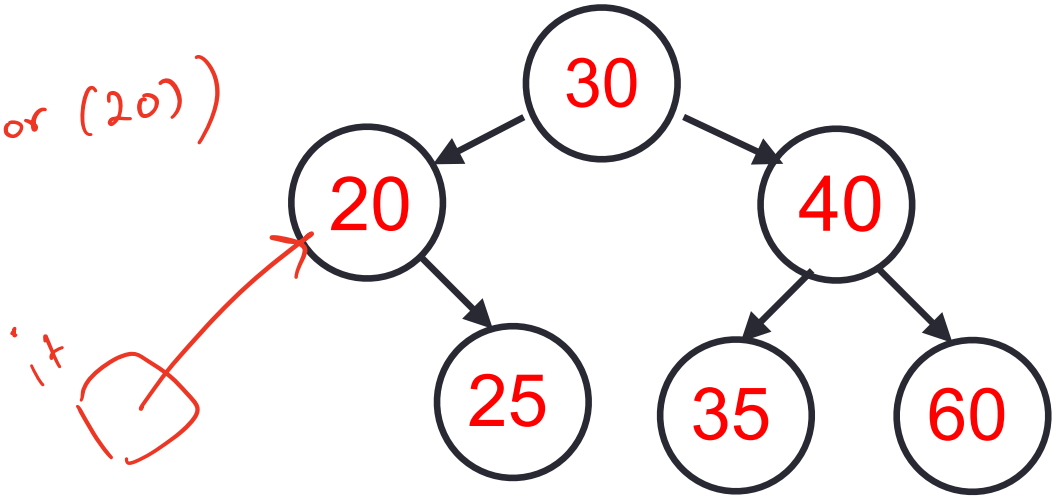
Same pattern works on different containers (set, vector, linked list…)

```
for (auto it = v.begin(); it != v.end(); ++it)  // vector

for (auto it = s.begin(); it != s.end(); ++it)  // set

for (auto it = m.begin(); it != m.end(); ++it)  // list
```

An iterator is an object that behaves like a pointer
– it points to an element in a container,
– you can dereference it and
– advance it.

Implement a new ADT called `iterator` that **abstracts a traversal pointer**!!!

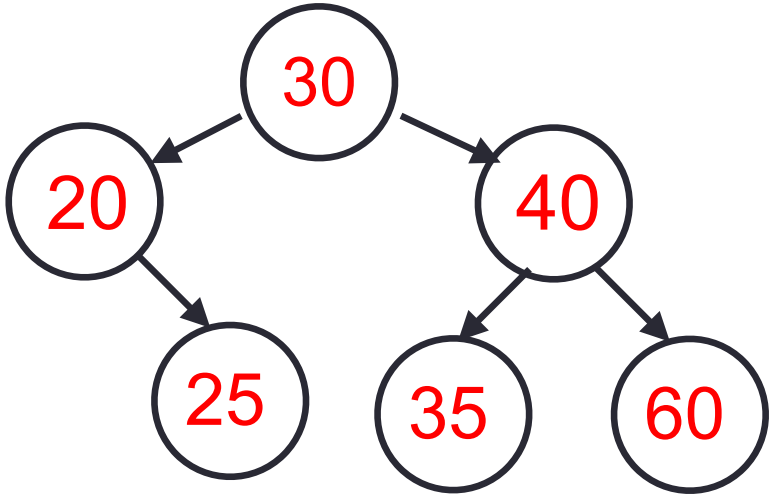
```
iterator it;
*it = 20 (data)
++it; // Moves to 25 (=successor(20))
```



Discuss (2 mins):
What functions does iterator class need to allow operations like `*it` and `++it`?

Big idea: Create a new ADT called `iterator` that behaves like a pointer.

```
class bst::iterator {
  public:



  private:

};
```




```
bst::iterator it;
*it = ____ (data)
++it; // Moves to ____
```

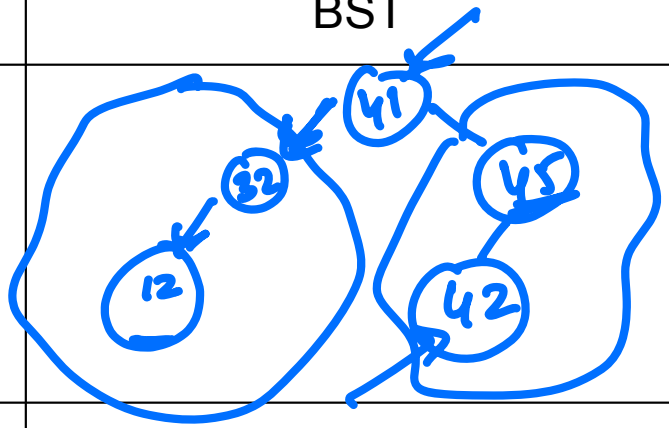
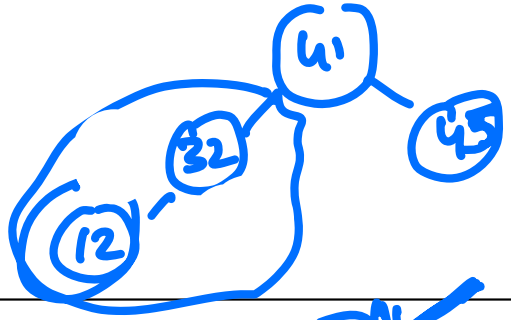
BST Helper functions to initialize iterators

Task 3.1: Implement `begin`: Returns an iterator to the smallest node.

```
bst::iterator bst::begin() {
    // Fill in the code




}
```

Task 3.2: Implement `end`: Returns an iterator for “past the end.”

```
bst::iterator bst::end() {
 // Fill in the code




}
```

Traversal algo	BST	Output
inorder(r: pointer to current node): if r is null: return inorder(r->left) process r (e.g., print r->val) inorder(r->right)		12, 32, 41, 42, 45 $T_L(41)$ 41, $T_R(41)$
postorder(r: pointer to current node): if r is null: return postorder(r->left) postorder(r->right) process r (e.g., print r->val)		12, 32, 45, 41 $T_L(41)$ $T_R(41)$ 41
preorder(r: pointer to current node): if r is null: return process r (e.g., print r->val) preorder(r->left) preorder(r->right)		41, 32, 12, 45 41 $T_L(41)$ $T_R(41)$

Post-order traversal: use to recursively clear the tree!

postorder(r : pointer to current node):
    if r is null, return
    postorder(r->left)
    postorder(r->right)
    process r (e.g., print r->val)

```
int bst::getHeight(Node *r) const{
    if (!r)
        return -1;
    int hleft = getHeight(r->left);
    int hright = getHeight(r->right);
    return max(hleft, hright) + 1;
}
```

Why would preorder not work for clear?

```
void bst::clear(Node *r){
    if (!r)
        return;
    clear(r->left);
    clear(r->right);
    delete r;
}
```

post order

When would you use each traversal and why?

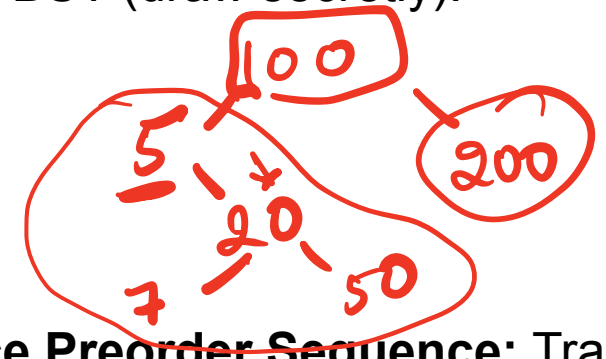
Inorder: useful when. . . printing in sorted order

Postorder: useful when. . . clearing the tree

Preorder: useful when. . . copy constructor

Pre-order traversal Game!

1. **Draw a BST:** Individually, draw a BST with 6 distinct keys (e.g., integers 1–10). Your
   Your BST (draw secretly):



100, 5, 20, 7, 50, 200

pre order traversal

100, 5, 20, 7, 50, 200

2. Trace Preorder Sequence: Trace the preorder traversal (root, left, right) of your BST and write the sequence of node values.

Your Preorder sequence:_____

(read the sequence to your partner, don't show them your BST diagram!)

Pre-order traversal Game!

3. Write the Pre-order sequence you received: 16, 0, -5, 1, 72, 100

4. Reconstruct the BST: Using your partner's sequence, rebuild their BST by inserting nodes in the given order, respecting BST properties.

5. Compare trees.

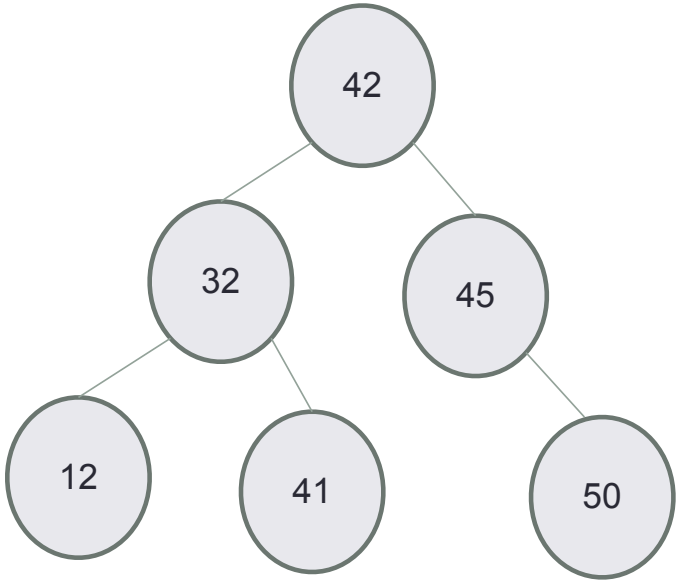
Leetcode problem

Given the `root` of a binary tree, return the preorder traversal of its nodes' values in a vector.

```
class Solution {
  public:
    vector<int> preorderTraversal(TreeNode* root) {}
};
```

https://leetcode.com/problems/binary-tree-preorder-traversal/description/?envType=problem-list-v2&envId=depth-first-search

Interview question: Write a function to extract BST keys into a vector. The extraction order must allow reconstructing the exact same tree structure by sequential insertion.



https://leetcode.com/problems/binary-tree-preorder-traversal/description/?envType=problem-list-v2&envId=depth-first-search

C++STL

- The C++ Standard Template Library is a handy set of three built-in components:
  - Containers: Data structures
  - Iterators: Standard way to traverse containers
  - Algorithms: These are what we ultimately use to solve problems

In this lecture, you learned how to implement an iterator for any custom ADT. Useful for working with STL classes and writing clean code in the upcoming assignment (PA01) where you have to implement a card game.
The big challenge is to iterate through the cards of two players in a seamless way (no passing around pointers like Node* in the main logic of your game). Use iterators!